

This homework focuses on the **Shortest Common Superstring (SCS)** problem, introduced during the lecture as a potential abstraction of the **Genome Assembly** problem.

Definition 1 (The SCS problem). *This stringology problem is defined as follows:*

Input. A set $\mathcal{S} = \{s_1, \dots, s_n\}$ of strings.

Output. A smallest sequence s^* such that $\forall s \in \mathcal{S}, s \subset s^*$ (every element of $\mathcal{S}$ appears in s^* as a substring).

(1) ■ The greedy heuristics

The greedy algorithm repeatedly chooses a pair of strings attaining the maximum overlap, and merges them until a single output string remains.

Algorithm 1: GreedySCS($\mathcal{S}$)

```
1:  $\mathcal{S} \leftarrow \text{list}(\mathcal{S})$ 
2: while  $|\mathcal{S}| > 1$  do
3:    $(u, v) \leftarrow \text{GETSTRINGSWITHMAXIMALOVERLAP}(\mathcal{S})$ 
4:    $w \leftarrow \text{MERGE}(u, v)$   $\triangleright$  (eg.  $\text{MERGE}(abc, bcde) = abcde$ )
5:    $\mathcal{S} \leftarrow (\mathcal{S} \setminus \{u, v\}) \cup \{w\}$ 
6: return  $\mathcal{S}[0]$ 
```

Despite its simplicity, the approximation ratio of this algorithm is unknown. It was conjectured by Tarhio and Ukkonen that it always achieves a 2-approximation in 1988. On September 2nd 2026, Shibata disproved the latter in an AI-assisted research paper, showing that the approximation ratio is at least $9/4$ by exhibiting a family of instances reaching this ratio.

► **Question 1.** Implement the greedy algorithm.

(2) ■ Attacking SCS as an Open-TSP problem

The *open* flavour of the **Travelling Salesman Problem (TSP)** asks for a minimal weight Hamiltonian path (not cycle!) on the input graph. While NP-complete, this problem benefits from very efficient heuristics that scale surprisingly well on real word instances. This turns its NP-completeness as an advantage: it reduces to other hard problems, and hence can help to practically solve their instances!

► **Question 2.** Propose a reduction from openTSP to the SCS problem, highlighting the graph construction and the superstring reconstruction.

❗ **Don't get stuck here:** discuss between each other and feel free to ask indications on Discord if needed!

► **Question 3.** Show that this reduction is correct when it holds that $\forall (u, v) \in \mathcal{S}, u \not\subset v$.

► **Question 4.** Implement this reduction, using `python-tsp*` as the underlying solver.

(3) ■ Solving the SCS problem on simulated reads

► **Question 5.** Simulate single-strain errorless reads from (prefixes of) `genome.fasta` using the function provided in `simreads.py`. How do the two approaches compare in terms of time and quality? How would you describe (if any) the impact of the read size of this analysis? Use figures to support your conclusions.



^{*}<https://github.com/fillipe-gsm/python-tsp>